트랜스포머:
어텐션으로 순서 데이터 읽기

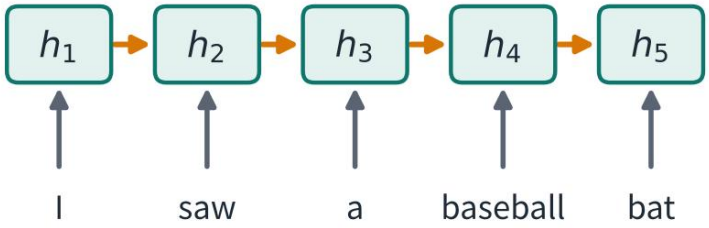
1 RNN의 한계와 어텐션 아이디어

기억을 전달하지 말고, 필요한 단어를 직접 보자

RNN의 두 가지 한계

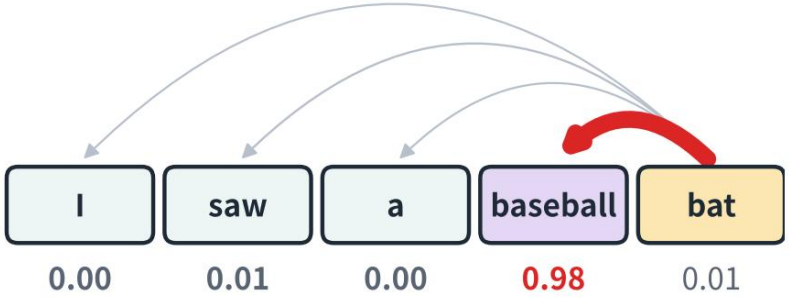
RNN: 기억 h를 한 칸씩 전달

baseball → bat까지 1칸, 멀면 여러 칸 거치며 흐려짐



h_2는 h_1이 끝나야 계산 → 순서대로만 (느림)

어텐션: bat이 모든 단어를 직접 봄



가중치 (실습1 학습 결과): baseball에 0.98 집중

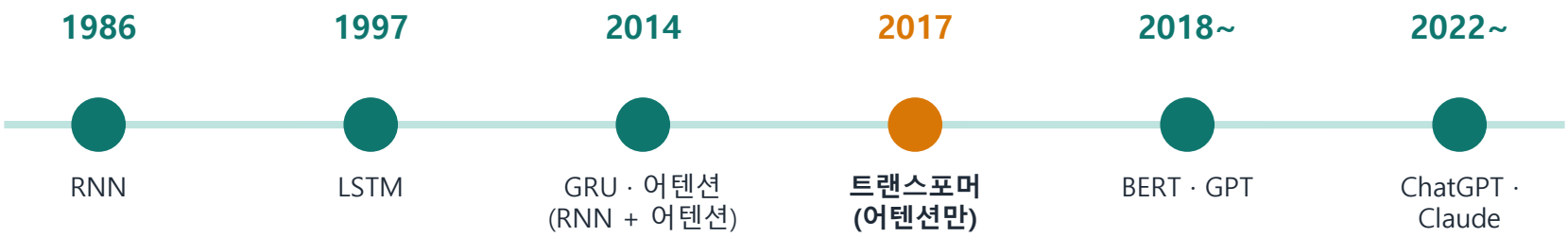
① 먼 정보가 흐려짐

50칸 전 단어는 h를 50번 거쳐야 도착 → 기울기 소실 (LSTM·GRU로 완화)

② 느림

h_t는 h_{t-1}이 있어야 계산 → GPU로 동시에 계산 못 함 → 큰 데이터 학습이 오래 걸림

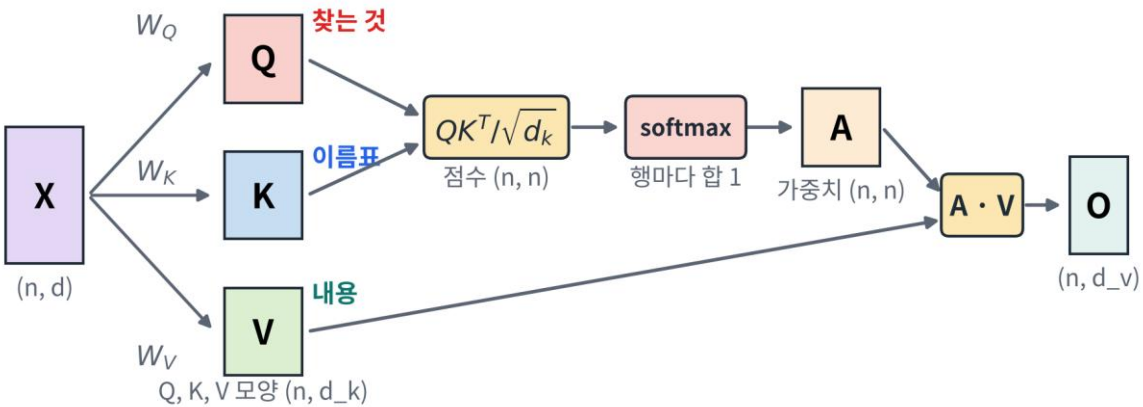
트랜스포머의 등장: Attention Is All You Need (2017)



	RNN · LSTM	트랜스포머
비유	귓속말 전달 게임: 앞사람 말을 한 명씩 전달, 멀수록 흐려짐	**단체 대화방: 모두의 말을 한 번에 보고 필요한 사람 말에 집중**
문맥 전달	기억 h를 다음 시점으로	**어텐션: 관련 있는 단어를 직접 찾아 섞음**
계산	순서대로 한 칸씩	**모든 단어 동시에 (병렬)**

어텐션의 재료: Query · Key · Value

이름	역할	도서관 비유	bat 예
Query (Q)	내가 찾는 것	검색어	bat: '내 뜻을 정해 줄 단어 어디 있지?'
Key (K)	나를 찾게 해 주는 이름표	책 제목	baseball: '나는 야구 관련'
Value (V)	실제로 전달할 내용	책 내용	baseball의 정보



$$\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q, K, V 만들기

같은 입력 X에 서로 다른 가중치를 곱함
Q = X·W_Q, K = X·W_K, V = X·W_V
W_Q, W_K, W_V는 학습으로 찾음

2 어텐션 계산

baseball bat을 손으로 계산해 보기

손계산 ①: 입력과 점수

단어	[야구, 동굴]	뜻
baseball	[1, 0]	야구 성질
cave	[0, 1]	동굴 성질
bat	**[1, 1]**	둘 다 (배트? 박쥐?)

설정

문장 [baseball, bat], W_Q = W_K = W_V = I (단위 행렬)
→ Q = K = V = X, d_k = 2

1을 √2 ≈ 1.41로 나눠서 0.71, 2를 나눠서 1.41

점수 = Q · Kᵀ / √2 (단어 쌍마다 얼마나 관련 있나)

Q ₩ K	baseball [1, 0]	bat [1, 1]
baseball [1, 0]	1·1 + 0·0 = 1 → 0.71	1·1 + 0·1 = 1 → 0.71
bat [1, 1]	**1·1 + 1·0 = 1 → 0.71**	**1·1 + 1·1 = 2 → 1.41**

내적 = 닮은 정도

두 벡터의 같은 칸끼리 곱해서 더함. 같은 방향일수록 큼. bat은 자기 자신과 가장 닮음(2), baseball과는 1.

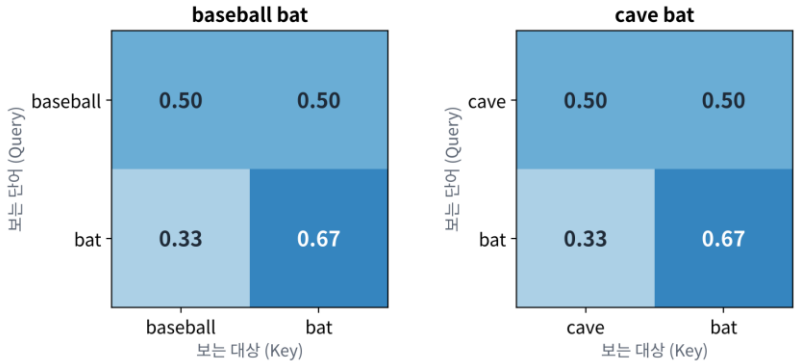
손계산 ②: softmax → 가중합

bat 행	baseball	bat
점수	0.71	1.41
e^x	2.03	4.11
softmax	**2.03 / 6.14 = 0.33**	**4.11 / 6.14 = 0.67**

bat 출력 = 0.33 × [1, 0] + 0.67 × [1, 1]
= [1.00, 0.67]

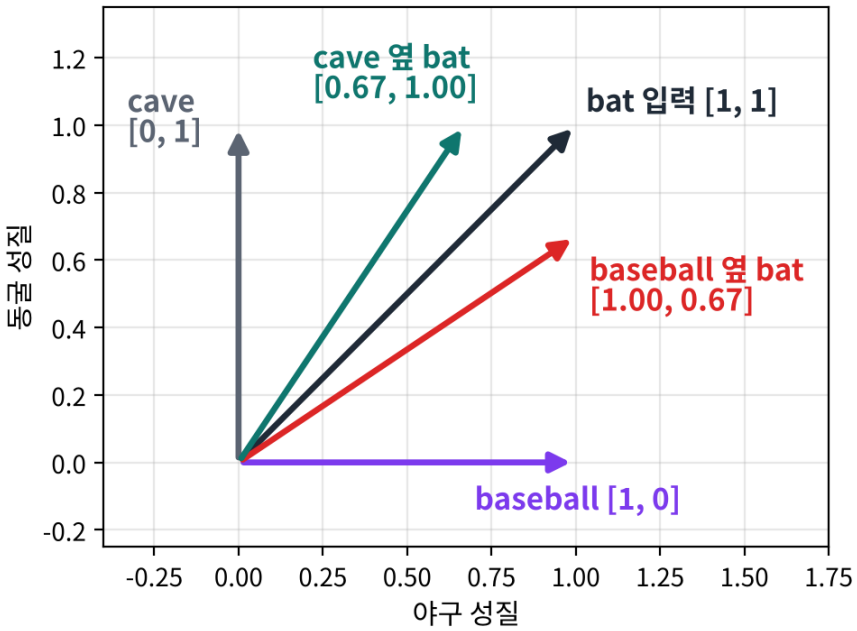
baseball 행

점수 [0.71, 0.71]로 같음 → 가중치 [0.50, 0.50]
출력 = 0.5×[1, 0] + 0.5×[1, 1] = [1.00, 0.50]



어텐션 맵: 행 = 보는 단어, 열 = 보는 대상. 행마다 합 1. 두 문장 모두 같은 숫자지만 섞이는 재료(V)가 다름 → 다음 장

같은 bat, 문맥에 따라 다른 벡터



문장	bat 입력	bat 출력
baseball bat	[1, 1]	**[1.00, 0.67]**
cave bat	[1, 1]	**[0.67, 1.00]**

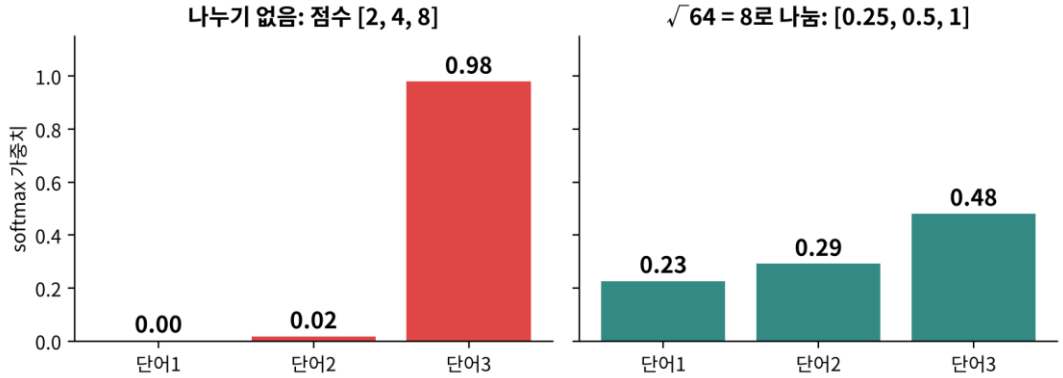
의미

입력은 같은 [1, 1]인데 어텐션을 지나면 baseball 옆에서는 야구 쪽, cave 옆에서는 동굴 쪽으로 기울어짐 → 다음 층이 배트/박쥐를 구분할 수 있음.

RNN 복습과 비교

RNN 실습1: W_hh(기억)가 0이면 배트·박쥐 구분 못 함 (0.50). 어텐션은 기억 없이 옆 단어를 직접 봄.

왜 $\sqrt{d_k}$로 나눌까



실습1 문제 3: 랜덤 벡터 64칸 → 나누기 없음 최댓값 1.00, 나누면 0.44

```
S = Q @ K.T / math.sqrt(d_k)    # 점수
A = torch.softmax(S, dim=-1)     # 행마다 합 1
O = A @ V                        # 가중합
```

나누지 않으면

벡터 칸이 64개면 내적이 수십까지 커짐 → softmax가 한 단어에 거의 1을 몰아줌 (0.98) → 나머지 단어의 기울기가 거의 0 → 학습이 안 됨

나누면

점수가 적당한 크기로 줄어 가중치가 고르게 퍼짐 (0.23 / 0.29 / 0.48) → 여러 단어를 보면서 학습

셀프 어텐션의 모양과 계산량

단계	모양	예: 배치 32, 단어 10, d = 64
입력 X	(배치, n, d)	(32, 10, 64)
Q, K, V	(배치, n, d_k)	(32, 10, 64)
점수 S, 가중치 A	**(배치, n, n)**	**(32, 10, 10): 단어 쌍마다 1개**
출력 O	(배치, n, d)	(32, 10, 64): 입력과 같은 모양

파라미터는 길이와 무관

W_Q, W_K, W_V는 (d, d)라 문장이 10단어든 1,000단어든 같음 (RNN의 W를 시점마다 재사용하는 것과 비슷)

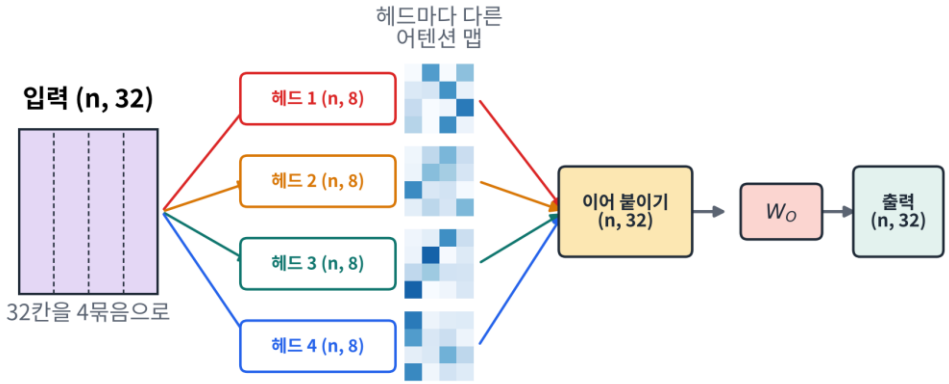
계산량은 n^2

단어 쌍마다 점수 → n = 10이면 100개, 1,000이면 1,000,000개. 긴 문장이 어려운 이유 (트랜스포머의 약점)

출력 = 입력 모양

(n, d) → (n, d) 그대로 → 블록을 여러 개 쌓기 쉬움. RNN의 out과 비슷하게 단어마다 벡터 1개

멀티헤드 어텐션: 여러 관점으로 동시에



왜 여러 개?

한 단어도 여러 관계가 있음
· 헤드 1: 주어-동사 (누가?)
· 헤드 2: 수식 관계 (어떤?)
· 헤드 3: 가까운 단어
헤드 하나로는 한 가지 가중치 분포만 가능

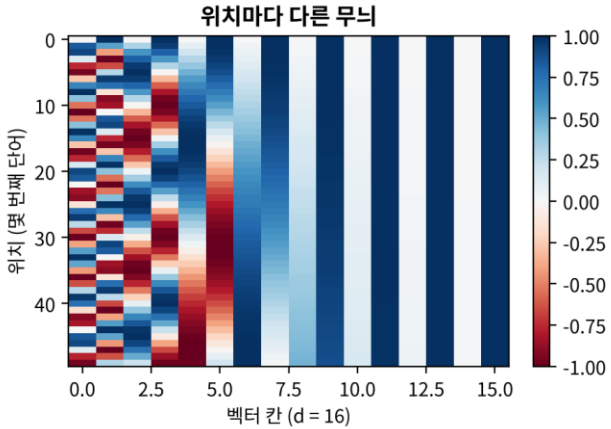
```
mha = nn.MultiheadAttention(embed_dim=32, num_heads=4,
                            batch_first=True)    # 헤드마다 32/4 = 8차원
out, attn = mha(x, x, x)     # 셀프 어텐션: Q, K, V 모두 x
# x (배치, n, 32) → out (배치, n, 32)
# embed_dim은 num_heads로 나눠떨어져야 함 (32 / 3 → 에러)
```

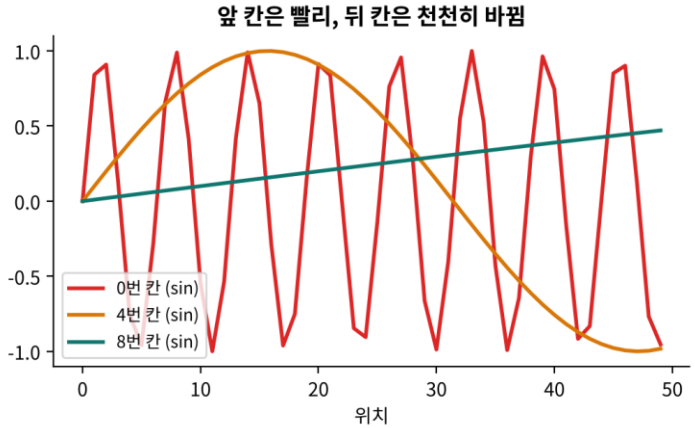
위치 인코딩: 순서 알려 주기

문장	bat 출력 (위치 정보 없음)	+ 위치 인코딩
baseball bat	[1.00, 0.67]	**[1.71, 1.46]**
bat baseball	**[1.00, 0.67] 똑같음**	**[1.16, 1.73]**

해결: 단어 벡터 + 위치 벡터

위치마다 다른 벡터를 만들어 단어 벡터에 더함.
· sin·cos 방식 (원래 논문, 아래 그림)
· 학습형: 위치 벡터도 가중치처럼 학습 (BERT, GPT, 실습2)





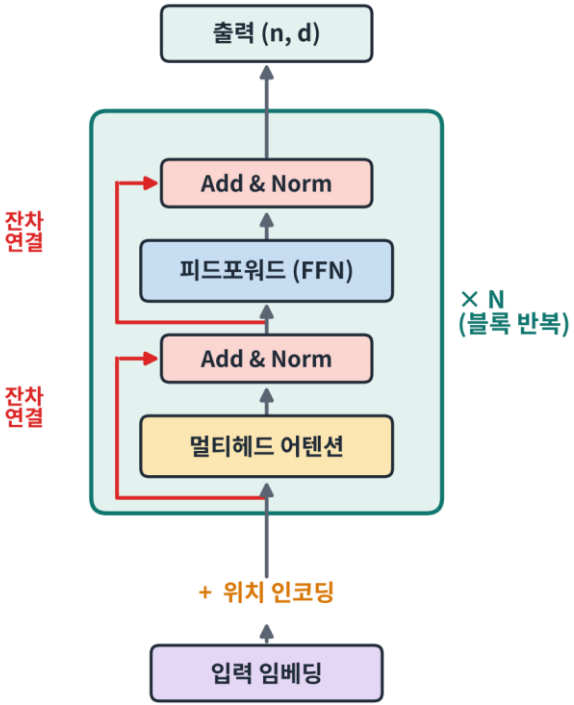
'개가 사람을 물었다' ≠ '사람이 개를 물었다'를 구별하려면 꼭 필요

3

트랜스포머 구조

어텐션 + FFN 블록을 쌓기

인코더 블록: 어텐션 + 피드포워드



부분	하는 일
멀티헤드 어텐션	단어끼리 정보 교환 (문맥 섞기)
피드포워드 (FFN)	단어마다 따로 Linear → ReLU → Linear로 가공
잔차 연결	입력을 출력에 그대로 더함: x + 어텐션(x). 기울기가 지나가는 지름길 (LSTM의 셀 상태 C와 비슷한 역할)
Add & Norm	더한 뒤 정규화 (LayerNorm): 값 크기를 일정하게 유지해 학습 안정

```
layer = nn.TransformerEncoderLayer(d_model=32, nhead=4,
          dim_feedforward=64, batch_first=True)
encoder = nn.TransformerEncoder(layer, num_layers=2)
h = encoder(x)     # (배치, n, 32) → (배치, n, 32)
```

인코더 · 디코더와 마스크



미래 가리기 마스크 (디코더): 지금 단어까지만 봄. 가린 칸 점수를 −∞로 → softmax 후 0

종류	마스크	잘하는 일	예
인코더만	없음: 앞뒤 모두 봄	이해: 분류, 감성 분석	BERT
디코더만	**미래 가림**	생성: 다음 단어 하나씩 만들기	GPT, Claude
인코더 + 디코더	디코더 쪽만 가림	변환: 번역, 요약	T5

왜 가리나?

다음 단어를 맞히는 학습에서 뒤 단어를 보면 정답을 보고 푸는 컨닝. 생성할 때도 뒤 단어는 아직 없음. (RNN은 원래 앞에서부터 읽어서 자연히 미래를 못 봄)

실습1 문제 6: bat 분류에 마스크를 씌우면 문맥 단어가 뒤에 있을 때 못 봐서 정확도 1.00 → 약 0.75

RNN · LSTM · 트랜스포머 비교

	RNN	LSTM · GRU	트랜스포머
문맥 전달	기억 h	h + 게이트 (+ 셀 상태 C)	**어텐션: 직접 보기**
먼 단어	흐려짐	더 오래 유지	**거리 상관없이 한 번에**
계산	순서대로	순서대로	**병렬 (빠름)**
순서 정보	구조에 들어 있음	구조에 들어 있음	위치 인코딩 필요
길이 n일 때 계산량	n에 비례	n에 비례	n^2에 비례
필요한 데이터	적어도 됨	적어도 됨	많을수록 강함
주로 쓰는 곳	짧은 시계열, 교육용	시계열, 센서, 음성	언어 모델, 번역, 이미지(ViT)

트랜스포머 = 어텐션(문맥) + FFN(가공) + 위치 인코딩(순서) + 잔차·정규화(깊게 쌓기)

이론 문제

문제 1. 손계산

단어 2개 a = [2, 0], b = [0, 2]
W_Q = W_K = W_V = I, d_k = 2
a 행의 (1) 점수 (2) 가중치 (3) 출력
(e^2.83 ≈ 17, e^0 = 1)

문제 2. 모양과 파라미터 수

nn.MultiheadAttention(embed_dim=64, num_heads=8, batch_first=True)
입력 (16, 20, 64)
(1) 출력 모양 (2) 헤드 하나의 차원
(3) 헤드별 어텐션 가중치 모양 (4) 파라미터 수 (편향 포함)

문제 3. 위치 인코딩

위치 인코딩 없이 '나는 너를 좋아해'와 '너를 나는 좋아해'를 넣으면 '좋아해'의 출력 벡터는 어떻게 될까? 이유는?

문제 4. 마스크

다음 중 미래 가리기 마스크가 필요한 것은?
(1) 영화 리뷰 → 긍정/부정
(2) 앞 문장 보고 다음 단어 생성 (챗봇)
(3) 문장 속 사람 이름 찾기

이론 정답

문제 1

(1) a·a = 4, a·b = 0 → /√2 → [2.83, 0]
(2) 17 / 18 = 0.94, 1 / 18 = 0.06
(3) 0.94×[2, 0] + 0.06×[0, 2] = [1.89, 0.11]

문제 2

(1) (16, 20, 64) (2) 64 / 8 = 8
(3) (16, 8, 20, 20)
(4) W_Q·W_K·W_V 3×(64×64 + 64) = 12,480
+ W_O 64×64 + 64 = 4,160 → 16,640

문제 3

똑같음. 어텐션은 단어들의 집합만 보고 가중합을 해서, 순서가 바뀌어도 같은 단어끼리 같은 점수 → 같은 출력. 위치 인코딩을 더해야 구별됨.

문제 4

(2)만. 생성은 앞 단어만 보고 다음 단어를 맞혀야 함.
(1)·(3)은 문장 전체를 이미 갖고 있어서 앞뒤를 모두 보는 게 유리 (인코더, BERT).

4 실습: 셀프 어텐션, 기온 예측

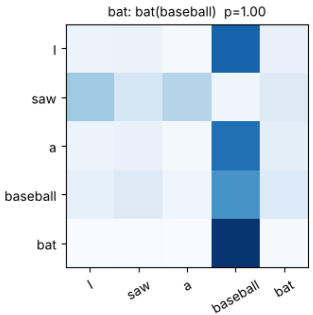
트랜스포머_실습1_셀프어텐션.ipynb · 트랜스포머_실습2_기온예측.ipynb

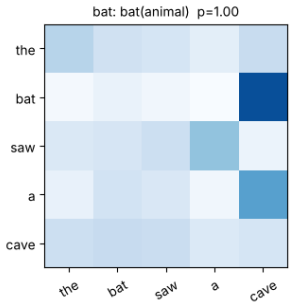
실습 1 셀프 어텐션 직접 계산하기

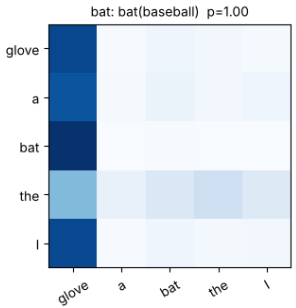
순서	내용	확인할 숫자
1	손계산 확인: Q·K·V → 점수 → softmax → 가중합	0.33 / 0.67
2	문맥에 따라 bat 벡터 변화	[1, 0.67] / [0.67, 1]
3	순서 바꾸기 → 위치 인코딩	같음 → 달라짐
4	nn.MultiheadAttention으로 같은 계산	1번과 같은 숫자
5	학습: bat → 배트? 박쥐?	정확도 1.000
6	마스크: 미래 가리기	대각선 위 0

5번 데이터

단어 5개 문장, bat + 문맥 단어 1개 + 의미 없는 단어
문맥: baseball·glove·pitcher → 배트
cave·night·wing → 박쥐
모델: 임베딩 → 셀프 어텐션 → bat 자리 출력 → Linear







학습 후 bat 행: 문맥 단어에 0.88 ~ 0.99 집중. 위치가 어디든 직접 찾아감

실습 2 기온 예측을 트랜스포머로: 모델

단계	코드	모양 (배치 64)
1 입력 임베딩	nn.Linear(1, 32)	(64, 30, 1) → (64, 30, 32)
2 + 위치 벡터 (학습형)	nn.Parameter(1, 30, 32)	(64, 30, 32)
3 인코더 블록 × 2	nn.TransformerEncoder	(64, 30, 32)
4 오늘 자리 → 예측	nn.Linear(32, 1)	(64, 32) → (64, 1)

```
class TempTransformer(nn.Module):
  def __init__(self, d_model=32, nhead=4, num_layers=2):
    super().__init__()
    self.inp = nn.Linear(1, d_model)
    self.pos = nn.Parameter(torch.zeros(1, 30, d_model))
    layer = nn.TransformerEncoderLayer(d_model, nhead,
              dim_feedforward=2*d_model, batch_first=True)
    self.encoder = nn.TransformerEncoder(layer, num_layers)
    self.fc = nn.Linear(d_model, 1)
  def forward(self, x):
    h = self.encoder(self.inp(x) + self.pos)
    return self.fc(h[:, -1, :])
```

RNN과 다른 점

· 기온 숫자 1개를 Linear로 32칸 벡터로 바꿔서 넣음 (단어라면 nn.Embedding)
· 순서를 모르니 위치 벡터를 더함
· 출력은 30일 모두 나오고, 그중 마지막 날(오늘) 자리만 씀 (RNN의 out[:, -1, :]와 같은 방식)

파라미터 18,145개 (RNN 1,153의 약 15배)

실습 2 결과: RNN 계열과 비교

기준선	RNN	LSTM	GRU	Transformer
2.02	1.74	1.77	1.76	1.76

test MAE (°C), 작을수록 좋음

실험: 30일 평균으로 예측	test MAE
위치 인코딩 있음	1.76 °C
위치 인코딩 없음	2.04 °C (기준선보다 나쁨)

위치 인코딩 실험

위치가 없으면 30일 중 어느 날이 오늘인지 모름 → 최근 30일 평균 수준만 맞힘. 위치를 넣으면 다시 1.76 °C.

해석

트랜스포머도 RNN 계열과 거의 같음. 이 데이터는 모델이 아니라 입력 정보(기온 1개)가 한계.

어텐션 맵도 30일에 거의 고르게 퍼짐 (0.029 ~ 0.039, 균등 0.033): bat처럼 꼭 봐야 할 한 날이 없는 데이터.

트랜스포머는 데이터가 많고 긴 문장·코드에서 강함.

정리

01	**아이디어**	기억 h를 한 칸씩 넘기지 않고, 필요한 단어를 직접 보고 섞음 (어텐션)
02	**Q · K · V**	찾는 것 · 이름표 · 내용. $\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$, W_Q·W_K·W_V는 학습
03	**멀티헤드**	차원을 나눠 여러 관점의 어텐션을 동시에, 이어 붙인 뒤 W_O
04	**위치 인코딩**	어텐션은 순서를 모름 → 위치 벡터를 더함 (sin·cos 또는 학습형)
05	**구조**	어텐션 + FFN + 잔차·정규화 블록을 쌓음. 디코더는 미래 가리기 마스크
06	**RNN과 비교**	병렬·먼 거리에 강함, 대신 n^2 계산과 많은 데이터 필요. 작은 시계열은 비슷